

DOCUMENTO DE DISEÑO (MINI GDD)

Miguel Pereira Ortiz

- **Motor usado:** Unity 2D
- **Requisitos cumplidos:** Todas las especificaciones de la entrega incluyendo extras (pooling, fragmentación...).

1. ESCENAS Y FLUJO DE JUEGO

El juego se estructura de forma fluida mediante una escena principal y elementos de interfaz superpuestos:

- **Escena Principal (Gameplay del juego):** Donde ocurre toda la acción, aparición de asteroides y sistemas de partículas.
- **Menú de pausa:** Interfaz que congela el tiempo (usando `Time.timeScale = 0`). Permite al jugador dos opciones: Continuar la partida o bien reiniciar la escena para intentar una nueva puntuación.
- **Pantalla de GAME OVER:** Se activa automáticamente al perder todas las vidas. Bloquea el input del jugador y presenta el resumen de la partida con opción de reiniciar la partida y volver a intentarlo.

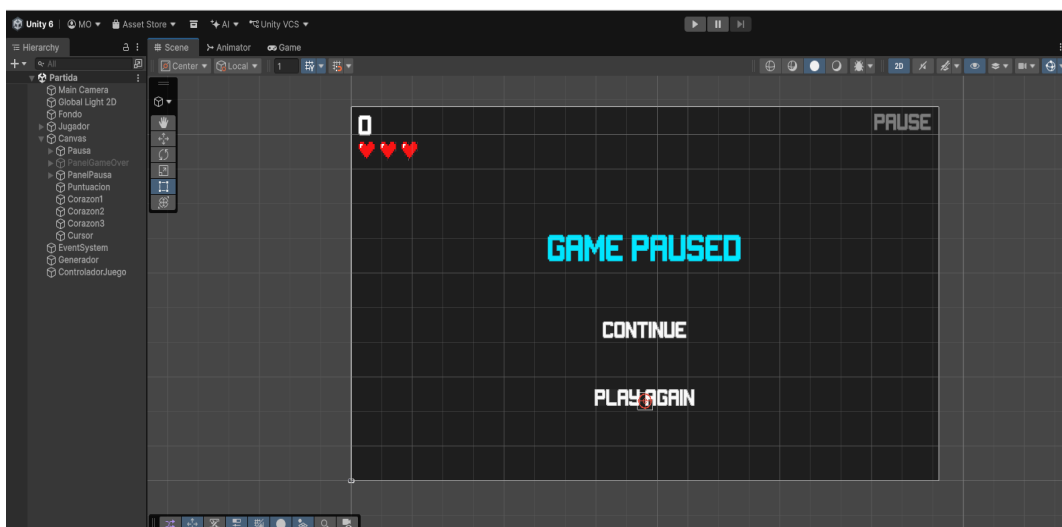


Imagen de la pantalla de Pausa y Hierarchy de la escena principal

2. ASSETS, GAMEOBJECTS Y EFECTOS VISUALES

En el apartado visual he priorizado el tiempo en lograr unas buenas partículas para dar la mejor sensación y feedback inmediato al jugador:

- **Nave del Jugador:** Personaje controlable principal. Incluye efectos de partículas de humo que se emiten dinámicamente al desplazarse por el espacio. Si el jugador sale de la pantalla por arriba, aparecerá por abajo y viceversa. Igual de derecha a izquierda.
- **Asteroides Padres (Grandes):** Asteroides algo más lentos con 5 puntos de vida. Cuentan con un sistema de Canvas UI para mostrar una barra de vida que cambia de color dinámicamente de **verde** a **rojo** según su porcentaje de salud.
- **Asteroides Hijos (Pequeños):** Unidades rápidas con uno de vida solamente. Se instancian en grupos de 4 al ser destruido un Asteroide Padre. Son algo más rápidos y pequeños, por lo que pueden llegar a ser muy molestos.
- **Cajas Sorpresa:** Cajas destructibles que contienen recompensas para la supervivencia, ya sea una vida extra o un modo de múltiple disparo.
- **Proyectiles:** Se distinguen visualmente entre balas estándar (blancas) y balas de power-up (verdes).
- **Partículas y Explosiones:** VFX al destruir asteroides o dispararlos o al recibir daño. También los asteroides tienen una animación en bucle del fuego que tiene por detrás, mejorando la experiencia del usuario.



Imagen del gameplay: Nave, Asteroides, Particle Effects, etc.

3. MECÁNICAS Y JUGABILIDAD

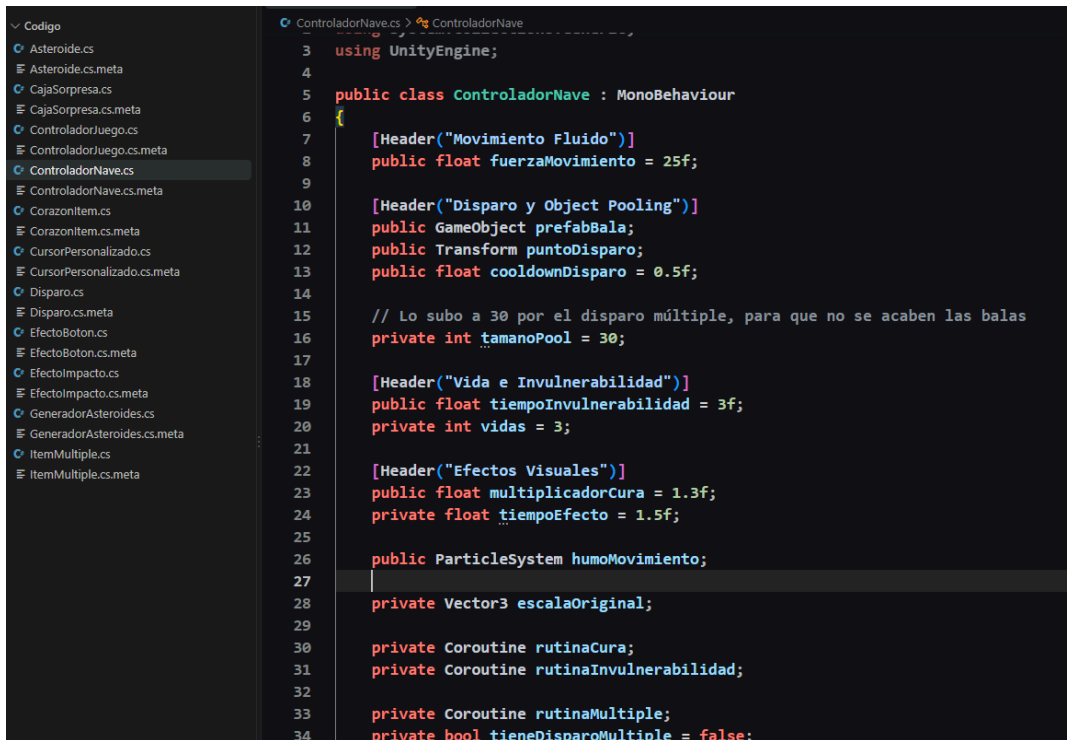
El juego se centra en la supervivencia, el movimiento ágil y la gestión táctica de los disparos:

- **Controles Base:** Desplazamiento multidireccional con WASD (o flechas). Apuntado de 360 grados usando la posición del puntero del ratón y disparo con el clic izquierdo.
- **Sistema de Enemigos (Fragmentación):** Al reducir a 0 la barra de vida de un Asteroide padre, este estalla y se divide en 4 asteroides hijos que son propulsados en las 4 direcciones cardinales, obligando al jugador a posicionarse rápidamente.
- **Sistema de Vidas y Puntuación:** UI constante que muestra la puntuación acumulada y las vidas restantes. Sufrir golpes reduce la vida hasta alcanzar el Game Over.
- **Mecánica de Cajas Sorpresa:** Al aparecer una caja, tiene una probabilidad aleatoria de ser uno de estos dos beneficios:
 - Corazón: Restaura vidas perdidas.
 - Disparo Múltiple: Transforma el disparo de la nave temporalmente. Se disparan 3 balas verdes simultáneas en forma de abanico, ampliando drásticamente la zona de impacto.

4. ARQUITECTURA DE SOFTWARE (SCRIPTS)

Para garantizar un rendimiento óptimo, especialmente con el disparo múltiple y las explosiones masivas, se ha diseñado una arquitectura escalable:

- **Patrón de Diseño (Object Pooling):** Sistema implementado para reciclar objetos. En lugar de crear (Instantiate) y destruir (Destroy) constantemente las balas verdes, fragmentos y partículas, se activan y desactivan de una "piscina" de objetos pre-instanciados. Esto evita caídas de FPS.
- **GameManager (ControladorJuego.cs):** Script principal que orquesta la puntuación, el estado de las vidas y la comunicación con la UI global.
- **PlayerController (ControladorNave.cs):** Procesa los inputs del usuario, aplica las físicas de movimiento y gestiona la cadencia del disparo.



```
3 using UnityEngine;
4
5 public class ControladorNave : MonoBehaviour
6 {
7     [Header("Movimiento Fluido")]
8     public float fuerzaMovimiento = 25f;
9
10    [Header("Disparo y Object Pooling")]
11    public GameObject prefabBala;
12    public Transform puntoDisparo;
13    public float cooldownDisparo = 0.5f;
14
15    // Lo subo a 30 por el disparo múltiple, para que no se acaben las balas
16    private int tamañoPool = 30;
17
18    [Header("Vida e Invulnerabilidad")]
19    public float tiempoInvulnerabilidad = 3f;
20    private int vidas = 3;
21
22    [Header("Efectos Visuales")]
23    public float multiplicadorCura = 1.3f;
24    private float tiempoEfecto = 1.5f;
25
26    public ParticleSystem humoMovimiento;
27
28    private Vector3 escalaOriginal;
29
30    private Coroutine rutinaCura;
31    private Coroutine rutinaInvulnerabilidad;
32
33    private Coroutine rutinaMultiple;
34    private bool tieneDisparoMultiple = false;
```

Imagen que muestra parte de código y el funcionamiento del pooling

5. CONTROLES E INTERACCIÓN

- Moverse: W, A, S, D o Flechas Direccionales
- Apuntar: Movimiento del Ratón
- Disparar: Clic Izquierdo del Ratón
- Pausar el juego: Botón de pausa
- Navegar Menús: Clic Izquierdo en botones de UI (Continuar, Reiniciar)

FIN DEL DOCUMENTO DEL DISEÑO